

ANN Model Architecture

Customer Churn Prediction — Stage 3 Deliverables

1. Overview

This document specifies the architecture of the Artificial Neural Network (ANN) model developed to predict customer churn. The model is implemented using scikit-learn's `MLPClassifier` and is trained on a preprocessed and scaled version of the customer dataset. The objective is to classify customers as likely to churn (1) or retain (0) based on 19 engineered input features derived from usage, demographic, and contract data.

2. Architecture Summary

The ANN follows a feedforward, fully connected architecture with three hidden layers of progressively decreasing size (128 → 64 → 32). This pyramid structure is a well-established pattern for binary classification tasks: the wider early layers capture broad feature interactions, while the narrower later layers compress this into a high-level representation for the output decision.

Layer	Type	Neurons / Units	Activation	Notes
Input Layer	Dense	19 features	—	Scaled features (StandardScaler)
Hidden Layer 1	Dense	128 neurons	ReLU	Primary feature extraction
Hidden Layer 2	Dense	64 neurons	ReLU	Pattern refinement
Hidden Layer 3	Dense	32 neurons	ReLU	High-level abstraction
Output Layer	Dense	1 neuron	Sigmoid	Binary churn probability

3. Layer-by-Layer Description

3.1 Input Layer

The input layer receives 19 numerical features corresponding to each customer record. Prior to training, all features are standardised using scikit-learn's `StandardScaler`, transforming each feature to zero mean and unit variance. This preprocessing step is critical for gradient-based

optimisers like Adam, ensuring that no single feature dominates the training signal due to scale differences.

- Number of input nodes: 19 (one per feature after one-hot encoding and feature engineering)
- Preprocessing: StandardScaler applied to both training and test sets
- Data split: 80% training / 20% testing (random_state=42 for reproducibility)

3.2 Hidden Layers

Three hidden layers are used, specified as hidden_layer_sizes=(128, 64, 32). All hidden layers use the ReLU (Rectified Linear Unit) activation function.

- Hidden Layer 1 (128 neurons): Captures primary feature interactions. The larger neuron count enables the network to explore a wide hypothesis space early in forward propagation.
- Hidden Layer 2 (64 neurons): Refines the learned representations from Layer 1, filtering irrelevant signal and strengthening key patterns.
- Hidden Layer 3 (32 neurons): Further compresses the representation into the most predictive abstract features before the output classification.

ReLU Activation: $f(x) = \max(0, x)$. ReLU introduces non-linearity without suffering from the vanishing gradient problem that affects sigmoid and tanh activations in deeper networks. It also enables sparse activation, improving computational efficiency.

3.3 Output Layer

The output layer consists of a single neuron with a Sigmoid (logistic) activation function, which is automatically applied by MLPClassifier for binary classification tasks. The sigmoid function squashes the output to a probability between 0 and 1, interpreted as the predicted probability of churn.

- Output: Continuous probability in [0, 1]
- Classification threshold: 0.5 (default — predicts churn if probability ≥ 0.5)
- Loss function: Binary cross-entropy (log loss)

4. Optimisation Configuration

The Adam optimiser (Adaptive Moment Estimation) is used for training. Adam combines the benefits of momentum-based gradient descent (which accelerates convergence) and RMSProp (which adapts learning rates per parameter). It is particularly well-suited for datasets with sparse gradients and noisy updates and consistently outperforms standard Stochastic Gradient Descent in classification tasks.

Hyperparameter	Value / Setting
Hidden Layer Sizes	(128, 64, 32)
Activation Function	ReLU (Rectified Linear Unit)
Solver / Optimiser	Adam (Adaptive Moment Estimation)
Maximum Iterations	500 epochs
Random State	42 (reproducibility)
Input Scaling	StandardScaler (zero mean, unit variance)
Output Activation	Logistic (Sigmoid) — auto for binary classification

5. Design Rationale

The architectural choices were guided by the following considerations:

- Pyramid structure (128 → 64 → 32): Progressively reduces dimensionality, forcing the network to learn increasingly compact and generalisable representations. This reduces overfitting compared to flat or expanding architectures.
- Three hidden layers: Sufficient depth to capture non-linear relationships between churn drivers without incurring the training instability of very deep networks on a tabular dataset of this size.
- ReLU activation: Avoids vanishing gradient issues and converges faster than sigmoid or tanh in hidden layers, which is particularly beneficial given the 500-epoch training budget.
- Adam solver: Requires minimal hyperparameter tuning and handles the class imbalance in churn datasets more robustly than vanilla SGD.
- max_iter=500: Provides sufficient iterations for the Adam optimiser to converge on this dataset size without excessive computation time.

6. Positioning Within Model Comparison

The ANN model was developed and evaluated alongside two benchmark models — Logistic Regression and Random Forest — to contextualise its predictive performance. The ANN is expected to outperform Logistic Regression on non-linear churn drivers (e.g., interaction effects between tenure and contract type) and to compete with Random Forest on overall accuracy and AUC, while offering a more interpretable probability output via the sigmoid activation.

Performance metrics collected for all three models include: Accuracy, Precision, Recall, F1 Score, and ROC-AUC. Comparative analysis is documented in the Final Report.